

```
pip install kagglehub
```

```
Requirement already satisfied: kagglehub in /usr/local/lib/python3.12/dist-packa
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packa
Requirement already satisfied: pyyaml in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packag
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-pa
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/d
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/d
```

```
import kagglehub
```

```
# Download latest version
```

```
path = kagglehub.dataset_download("kazanov/sentiment140")
```

```
print("Path to dataset files:", path)
```

```
Warning: Looks like you're using an outdated `kagglehub` version (installed: 0.3
Downloading from https://www.kaggle.com/api/v1/datasets/download/kazanov/sentiment140
100%|██████████| 80.9M/80.9M [00:04<00:00, 18.5MB/s]Extracting files...
```

```
Path to dataset files: /root/.cache/kagglehub/datasets/kazanov/sentiment140/ver
```

```
import os
```

```
import pandas as pd
```

```
# List files in downloaded folder
```

```
print(os.listdir(path))
```

```
['training.1600000.processed.noemoticon.csv']
```

```
file_path = os.path.join(path, "training.1600000.processed.noemoticon.csv")
```

```
columns = ['target', 'ids', 'date', 'flag', 'user', 'text']
```

```
df = pd.read_csv(file_path, encoding='latin-1', names=columns)
```

```
df.head()
```

	target	ids	date	flag	user	text
0	0	1467810369	Mon Apr 06 22:19:45 PDT 2009	NO_QUERY	_TheSpecialOne_	@switchfoot - Awww, t...
1	0	1467810672	Mon Apr 06 22:19:49 PDT 2009	NO_QUERY	scotthamilton	is upset that he can't update his Facebook by ...
2	0	1467810917	Mon Apr 06 22:19:53 PDT 2009	NO_QUERY	mattycus	@Kenichan I dived many times for the ball. Man...
			Mon Apr			

```
df['sentiment'] = df['target'].map({0: "negative", 4: "positive"})
df.head()
```

	target	ids	date	flag	user	text
0	0	1467810369	Mon Apr 06 22:19:45 PDT 2009	NO_QUERY	_TheSpecialOne_	@switchfoot http://twitpic.com/2y1zl - Awww, t...
1	0	1467810672	Mon Apr 06 22:19:49 PDT 2009	NO_QUERY	scotthamilton	is upset that he can't update his Facebook by ...
2	0	1467810917	Mon Apr 06 22:19:53 PDT 2009	NO_QUERY	mattycus	@Kenichan I dived many times for the ball. Man...
3	0	1467811184	Mon Apr 06 22:19:57 PDT 2009	NO_QUERY	ElleCTF	my whole body feels itchy and like its on fire
4	0	1467811193	Mon Apr 06 22:19:57 PDT 2009	NO_QUERY	Karoli	@nationwideclass no, it's not behaving at all....

```

import re
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize

stop_words = set(stopwords.words('english'))

def preprocess(text):
    text = text.lower()
    text = re.sub(r"http\S+|@\w+|#\w+", "", text) # remove URLs, mentions, has
    text = re.sub(r"^[a-zA-Z\s]", "", text) # remove special chars
    tokens = word_tokenize(text)
    tokens = [word for word in tokens if word not in stop_words]
    return " ".join(tokens)

df['clean_text'] = df['text'].apply(preprocess)

```

```

from textblob import TextBlob

```

```
def get_sentiment(text):
    polarity = TextBlob(text).sentiment.polarity
    if polarity > 0:
        return "positive"
    elif polarity < 0:
        return "negative"
    else:
        return "neutral"

df['predicted_sentiment'] = df['clean_text'].apply(get_sentiment)
```

```
from sklearn.metrics import classification_report

print(classification_report(df['sentiment'], df['predicted_sentiment']))
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565:
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565:
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```

	precision	recall	f1-score	support
negative	0.75	0.31	0.44	800000
neutral	0.00	0.00	0.00	0
positive	0.64	0.54	0.59	800000
accuracy			0.42	1600000
macro avg	0.46	0.28	0.34	1600000
weighted avg	0.69	0.42	0.51	1600000

```
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565:
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```

```
tweet = "I love this new phone! The battery life is amazing."
cleaned = preprocess(tweet)
print(get_sentiment(cleaned))
```

positive